

Assignment tasks



Submitted from:

Hina Bashir(BCS-F24-E04)

Semester 5th (self)

Submitted to:

Mr.Raza

Operating system

Code No#1

```
#include <iostream>
#include <thread>
#include <vector>
#include <windows.h>

HANDLE sem;

void worker(int id) {

    WaitForSingleObject(sem, INFINITE);
    std::cout << "Thread " << id << " acquired resource.\n";

    Sleep(1000);

    std::cout << "Thread " << id << " releasing resource.\n";

    ReleaseSemaphore(sem, 1, nullptr);
}

int main() {

sem = CreateSemaphoreA(nullptr, 2, 2, nullptr);
std::vector<std::thread> threads;
    for (int i = 1; i <= 4; ++i) {
        threads.emplace_back(worker, i);
    }
    WaitForSingleObject(sem, INFINITE);
    std::cout << "Thread " << id << " acquired resource.\n";

    Sleep(1000);

    std::cout << "Thread " << id << " releasing resource.\n";

    ReleaseSemaphore(sem, 1, nullptr);
}

int main() {

sem = CreateSemaphoreA(nullptr, 2, 2, nullptr);
std::vector<std::thread> threads;
    for (int i = 1; i <= 4; ++i) {
        threads.emplace_back(worker, i);
    }

    for (auto& t : threads) {
        t.join();
    }

    CloseHandle(sem);
    return 0;
}
```

Output:

```

C:\Users\NEW AL-MADINA\Documents\opera.exe
Thread 4 acquired resource.
Thread 1 acquired resource.
Thread 4 releasing resource.
Thread 1 releasing resource.
Thread 2 acquired resource.
Thread 3 acquired resource.
Thread 2 releasing resource.
Thread 3 releasing resource.
Process exited after 2.041 seconds with return value 0
Press any key to continue . . .

```

Code#2

```

#include <iostream>
#include <windows.h>

int main() {

    void* base = VirtualAlloc(nullptr, 1024 * 1024,
    MEM_RESERVE, PAGE_NOACCESS);
    if (!base) return 1;
    std::cout << "Reserved 1 MB address space at: " << base << "\n";

    int* page = (int*)VirtualAlloc(base, 4096,
    MEM_COMMIT, PAGE_READWRITE);
    if (!page) return 1;
    page[0] = 42;
    std::cout << "Committed memory value: " << page[0] << "\n";

    VirtualFree(base, 0, MEM_RELEASE);
    return 0;
}

```

Output:

```

C:\Users\NEW AL-MADINA\Documents\Untitled6.exe
Reserved 1 MB address space at: 0xc30000
Committed memory value: 42
Process exited after 0.01508 seconds with return value 0
Press any key to continue . . .

```

Code#3

```

#include <iostream>
#include <cstring>
#include <windows.h>

int main() {

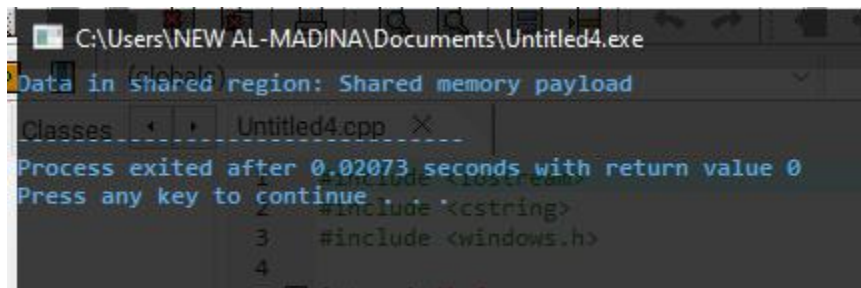
    HANDLE hMap = CreateFileMappingA(INVALID_HANDLE_VALUE, nullptr,
    PAGE_READWRITE, 0, 256, "Local\\MySharedMem");
    if (!hMap) return 1;

    char* pBuf = (char*)MapViewOfFile(hMap, FILE_MAP_ALL_ACCESS, 0, 0, 256);
    if (!pBuf) {
        CloseHandle(hMap);
        return 1;
    }

    strcpy_s(pBuf, 256, "Shared memory payload");
    std::cout << "Data in shared region: " << pBuf << "\n";
    UnmapViewOfFile(pBuf);
    CloseHandle(hMap);
    return 0;
}

```

Output:



```

C:\Users\NEW AL-MADINA\Documents\Untitled4.exe
Data in shared region: Shared memory payload
Process exited after 0.02073 seconds with return value 0
Press any key to continue

```

Code#4

```

#include <iostream>
#include <thread>
#include <windows.h>

void clientWorker() {
    Sleep(500);
    HANDLE hClient = CreateFileA("\\\\.\\pipe\\TestPipe",
    GENERIC_WRITE, 0, nullptr, OPEN_EXISTING, 0, nullptr);
    if (hClient != INVALID_HANDLE_VALUE) {
        const char msg[] = "Hello through Named Pipe!";
        DWORD written = 0;
        WriteFile(hClient, msg, sizeof(msg), &written, nullptr);
        CloseHandle(hClient);
    }
}

int main() {
    HANDLE hServer = CreateNamedPipeA("\\\\.\\pipe\\TestPipe",
    PIPE_ACCESS_INBOUND,
    PIPE_TYPE_MESSAGE | PIPE_READMODE_MESSAGE | PIPE_WAIT, 1, 512, 512, 0, nullptr);

    std::thread client(clientWorker);
    ConnectNamedPipe(hServer, nullptr);
    char buffer[128] = {};
    DWORD readBytes = 0;
    ReadFile(hServer, buffer, sizeof(buffer) - 1, &readBytes, nullptr);
    std::cout << "Server received: " << buffer << "\n";

    client.join();
    DisconnectNamedPipe(hServer);
    CloseHandle(hServer);
    return 0;
}

```

Output:

```

C:\Users\NEW AL-MADINA\Documents\Untitled5.exe
11  DWORD written = 0;
Server received: Hello through Named Pipe!
12  ReadFile(hClient, buf, sizeof(buf), &written, nullptr);
13  CloseHandle(hClient);
-----
Process exited after 0.5316 seconds with return value 0
Press any key to continue . . .
16  int main() {
17  HANDLE hServer = CreateNamedPipeA("\\\\.\\pipe\\TestPi

```

Code#5

```

#include <iostream>
#include <windows.h>

int main() {

void* base = VirtualAlloc(nullptr, 1024 * 1024,
MEM_RESERVE, PAGE_NOACCESS);
if (!base) return 1;
std::cout << "Reserved 1 MB address space at: " << base << "\n";

int* page = (int*)VirtualAlloc(base, 4096,
MEM_COMMIT, PAGE_READWRITE);
if (!page) return 1;

page[0] = 42; |
std::cout << "Committed memory value: " << page[0] << "\n";

VirtualFree(base, 0, MEM_RELEASE);
return 0;
}

```

Output:

```

C:\Users\NEW AL-MADINA\Documents\Untitled5.exe
11  DWORD written = 0;
Server received: Hello through Named Pipe!
12  ReadFile(hClient, buf, sizeof(buf), &written, nullptr);
13  CloseHandle(hClient);
-----
Process exited after 0.5316 seconds with return value 0
Press any key to continue . . .
16  int main() {
17  HANDLE hServer = CreateNamedPipeA("\\\\.\\pipe\\TestPi

```

Code#6

```

#include <iostream>
#include <windows.h>

int main() {

void* base = VirtualAlloc(nullptr, 1024 * 1024,
MEM_RESERVE, PAGE_NOACCESS);
if (!base) return 1;
std::cout << "Reserved 1 MB address space at: " << base << "\n";

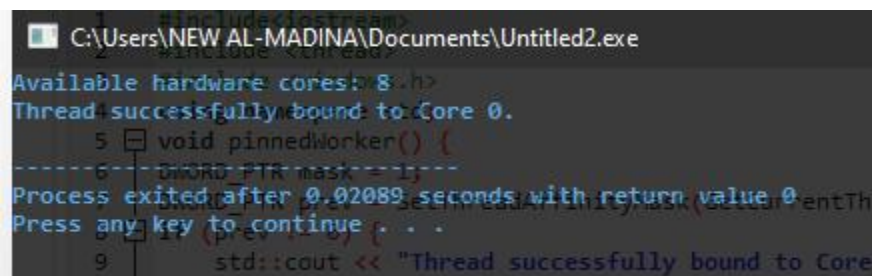
int* page = (int*)VirtualAlloc(base, 4096,
MEM_COMMIT, PAGE_READWRITE);
if (!page) return 1;

page[0] = 42; |
std::cout << "Committed memory value: " << page[0] << "\n";

VirtualFree(base, 0, MEM_RELEASE);
return 0;
}

```

Output:



```

C:\Users\NEW AL-MADINA\Documents\Untitled2.exe
Available hardware cores: 8
Thread successfully bound to Core 0.
Process exited after 0.02089 seconds with return value 0
Press any key to continue

```

Code#7

```

#include <iostream>
#include <future>
#include <chrono>
#include <thread>

int main() {

std::future<double> task = std::async(std::launch::async, []() {
std::this_thread::sleep_for(std::chrono::seconds(1));
return 3.14159;
});

std::cout << "Main thread continues work while async task computes...\n";

|
double result = task.get();
std::cout << "Result received: " << result << "\n";
return 0;
}

```

Output:

```
1 #include <iostream>
2 #include <future>
3
4 Main thread continues work while async task computes...
5 Result received: 3.14159
6
7 -----
8 int main() {
9     Process exited after 1.056 seconds with return value 0
10    Press any key to continue
11    std::future<double> task = std::async(std::launch::
12    std::this_thread::sleep_for(std::chrono::seconds(1
13    return 3.14159;
14    });
15 }
```

Code#8

```
#include <iostream>
#include <windows.h>

int main() {
    int x = 10;
    MEMORY_BASIC_INFORMATION mbi{};

    if (VirtualQuery(&x, &mbi, sizeof(mbi)) != 0) {
        std::cout << "Address: " << mbi.BaseAddress << "\n";
        std::cout << "Region Size: " << mbi.RegionSize << " bytes\n";
        std::cout << "State: " << (mbi.State == MEM_COMMIT ? "MEM_COMMIT" : "OTHER") << "\n";
    }
    return 0;
}
```

Output:

```
1 (info) C:\Users\NEW AL-MADINA\Documents\sdf.exe
2 Address: 0x78f000
3 Region Size: 4096 bytes
4 State: MEM_COMMIT
5 -----
6 Process exited after 0.02477 seconds with return value 0
7 Press any key to continue
8
9 int main() {
10    int x = 10;
11    MEMORY_BASIC_INFORMATION mbi{};
```

Code#9

```

#include <iostream>
#include <windows.h>

int main() {
    volatile long count = 0;
    for (long i = 0; i < 200000000; ++i) { count += i; }

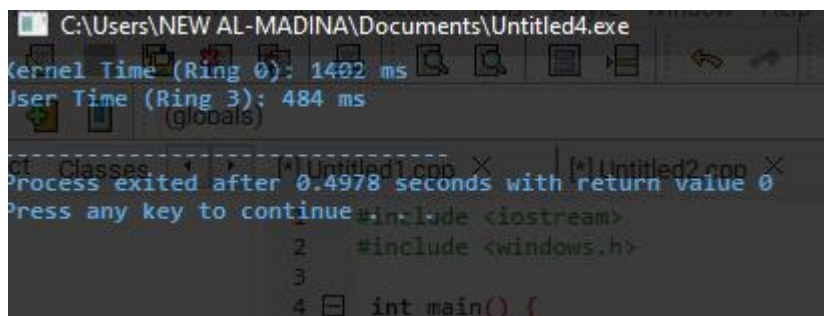
    FILETIME createTime, exitTime, kernelTime, userTime;
    GetProcessTimes(GetCurrentProcess(),
        &createTime, &exitTime, &kernelTime, &userTime);

    ULARGE_INTEGER kTime, uTime;
    uTime.LowPart = kernelTime.dwLowDateTime;
    kTime.HighPart = kernelTime.dwHighDateTime;
    uTime.LowPart = userTime.dwLowDateTime;
    uTime.HighPart = userTime.dwHighDateTime;

    // Convert 100-nanosecond units to milliseconds (/ 10,000)
    std::cout << "Kernel Time (Ring 0): "
    << kTime.QuadPart / 10000 << " ms\n";
    std::cout << "User Time (Ring 3): "
    << uTime.QuadPart / 10000 << " ms\n";
    return 0; }

```

Output:



```

C:\Users\NEW AL-MADINA\Documents\Untitled4.exe
Kernel Time (Ring 0): 1402 ms
User Time (Ring 3): 484 ms
(globals)
Classes
Process exited after 0.4978 seconds with return value 0
Press any key to continue.
1 #include <iostream>
2 #include <windows.h>
3
4 int main() {

```

Code:

```

#include <iostream>
#include <thread>
#include <vector>
#include <atomic>

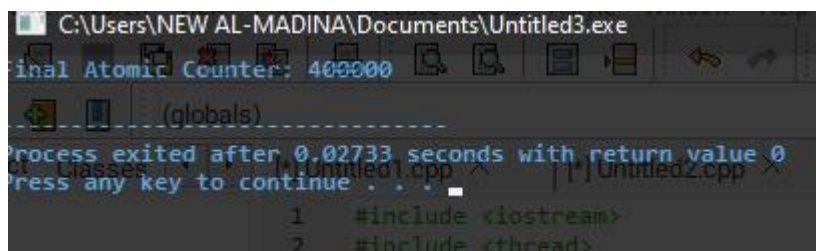
std::atomic<long> counter{0};

void increment() {
    for (int i = 0; i < 100000; ++i) {
        // Hardware-enforced uninterrupted increment
        counter.fetch_add(1, std::memory_order_relaxed);
    }
}

int main() {
    std::vector<std::thread> threads;
    for (int i = 0; i < 4; ++i) {
        threads.emplace_back(increment);
    }
    for (auto& t : threads) {
        t.join();
    }
    std::cout << "Final Atomic Counter: " << counter << "\n";
    return 0;
}

```

Output:



```

C:\Users\NEW AL-MADINA\Documents\Untitled3.exe
Final Atomic Counter: 400000
(globals) -----
Process exited after 0.02733 seconds with return value 0
Press any key to continue . . .

```

Code

```
#include <iostream>
#include <thread>
#include <mutex>
#include <chrono>
```

```
std::mutex m1, m2;
```

```
void threadA() {
    m1.lock();
    std::cout << "Thread A locked Mutex 1, waiting for Mutex 2...\n";
    std::this_thread::sleep_for(std::chrono::milliseconds(50));
    m2.lock();
    m2.unlock();
    m1.unlock();
}
```

```
void threadB() {
    m2.lock();
    std::cout << "Thread B locked Mutex 2, waiting for Mutex 1...\n";
    std::this_thread::sleep_for(std::chrono::milliseconds(50));
    m1.lock();
    m1.unlock();
    m2.unlock();
}
```

```
void threadA() {
    m1.lock();
    std::cout << "Thread A locked Mutex 1, waiting for Mutex 2...\n";
    std::this_thread::sleep_for(std::chrono::milliseconds(50));
    m2.lock();
    m2.unlock();
    m1.unlock();
}
```

```
void threadB() {
    m2.lock();
    std::cout << "Thread B locked Mutex 2, waiting for Mutex 1...\n";
    std::this_thread::sleep_for(std::chrono::milliseconds(50));
    m1.lock();
    m1.unlock();
    m2.unlock();
}
```

```
int main() {
    std::thread t1(threadA);
    std::thread t2(threadB);
    t1.join();
    t2.join();
    return 0;
}
```

Output:

```
C:\Users\NEW AL-MADINA\Documents\Untitled2.exe
Thread A locked Mutex 1, waiting for Mutex 2x.1, wait
Thread B locked Mutex 2, waiting for Mutex 1: chrono::mi
12 m2.lock(); // Blocks waiting for threadB to
13 m2.unlock();
14 m1.unlock();
15 }
16
```

Code

```
#include <iostream>
#include <windows.h>

int main() {
    STARTUPINFOA si{};
    PROCESS_INFORMATION pi{};
    si.cb = sizeof(si);

    char command[] = "cmd.exe /C exit 42";

    if (CreateProcessA(nullptr, command, nullptr, nullptr,
        FALSE, 0, nullptr, nullptr, &si, &pi)) {
        WaitForSingleObject(pi.hProcess, INFINITE);

        DWORD exitCode = 0;
        GetExitCodeProcess(pi.hProcess, &exitCode);
        std::cout << "Child process exited with code: " << exitCode << "\n";

        CloseHandle(pi.hThread);
        CloseHandle(pi.hProcess);
    }
    return 0;
}
```

Output:

```
Select C:\Users\NEW AL-MADINA\Documents\Untitled1.exe
Child process exited with code: 42
Classes Untitled1.cpp x
Process exited after 0.05122 seconds with return value 0
Press any key to continue
#include <windows.h>
3
4 int main() {
```